

Projetos em Ambientes Web e Cloud

Licenciatura em Tecnologias Digitais e
Inteligência Artificial

Aula 3

João Matos

joao_miguel_goncalves_matos@iscte-iul.pt

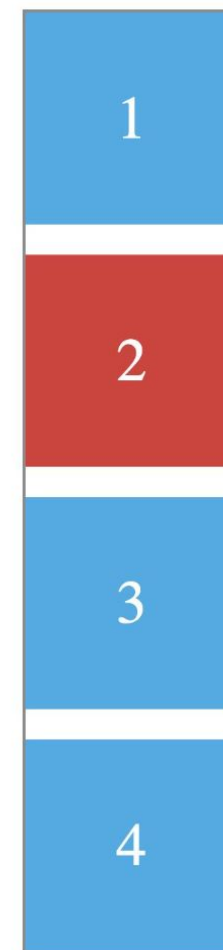
2º Semestre

CSS Position - static

- Posição inicial de cada elemento, ignora atributos como o top, bottom, left ou right;
- Um elemento com `position: static;` não está posicionado de maneira especial. Fica apenas posicionado de acordo com o fluxo normal da página.

```
.box {  
  position: relative;  
  display: inline-flex;  
  border: #8B8B8B solid 1px;  
  flex-direction: column;  
}
```

```
.static {  
  position: static;  
  background-color: #db3636;  
  top: 15px;  
  left: 15px;  
}
```

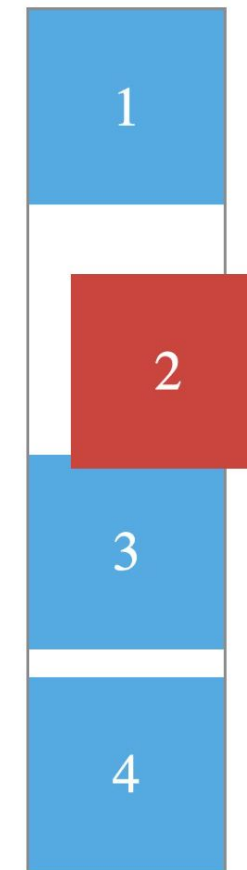


CSS Position - relative

- O elemento é posicionado relativamente à sua própria posição;
- Os atributos `top`, `bottom`, `left` e `right`, terão efeito na posição final do elemento, mas os outros elementos não serão afetados;

```
.box {  
  position: relative;  
  display: inline-flex;  
  border: #8B8B8B solid 1px;  
  flex-direction: column;  
}
```

```
.relative {  
  position: relative;  
  background-color: #db3636;  
  top: 15px;  
  left: 15px;  
}
```



CSS Position - *fixed*

- O elemento é posicionado relativamente à janela;
- Os elementos são removidos do fluxo da página, pelo que o espaço que ocupavam é libertado;

```
.fixed {  
  position: fixed;  
  background-color: #db3636;  
  top: 15px;  
  right: 15px;  
}
```

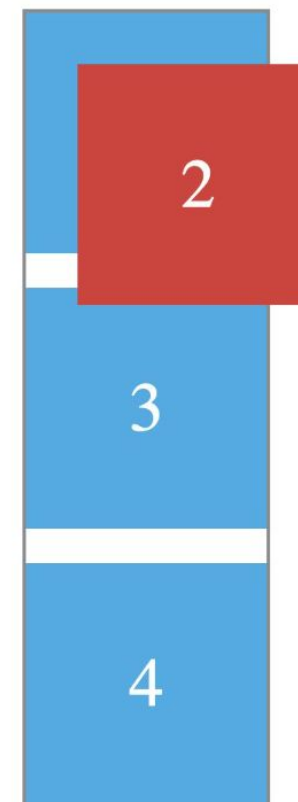


CSS Position - absolute

- O elemento é posicionado relativamente ao seu “parente”
- Os elementos são removidos do fluxo da página, por isso os elementos podem ficar sobrepostos (*não ocupam espaço no layout*)

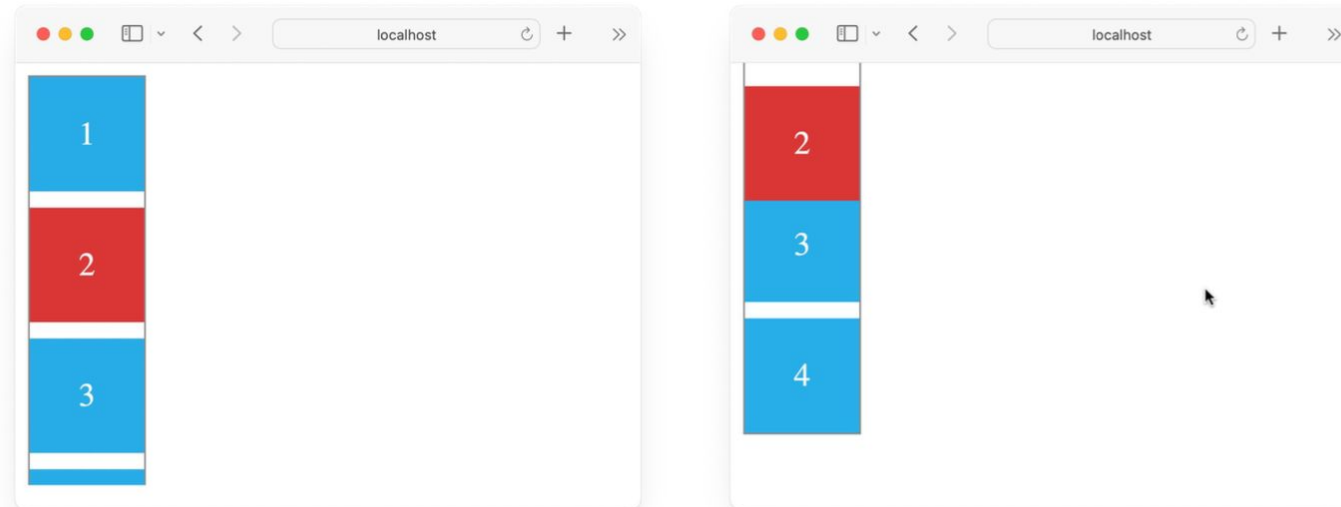
```
.box {  
  position: relative;  
  display: inline-flex;  
  border: #8B8B8B solid 1px;  
  flex-direction: column;  
}
```

```
.absolute {  
  position: absolute;  
  background-color: #db3636;  
  top: 15px;  
  left: 15px;  
}
```



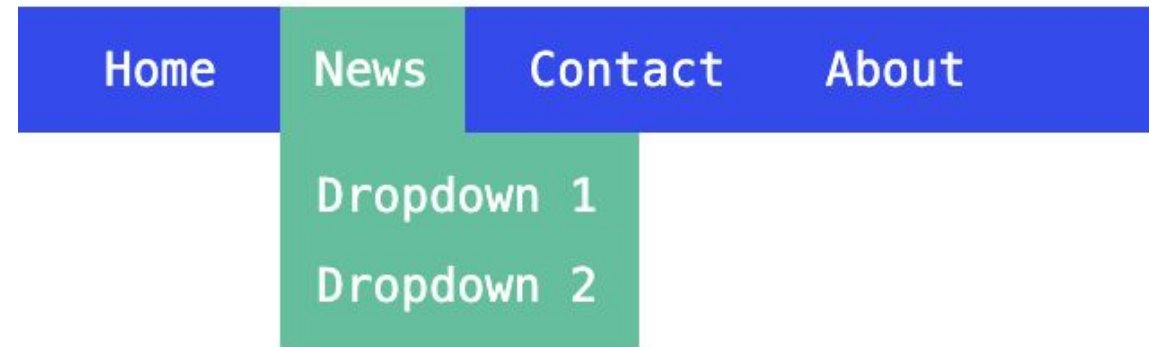
CSS Position - sticky

- O elemento tem comportamento de posição relativa até chegar ao limite da janela ao fazermos scroll. Depois muda para um comportamento de posição fixa.



Tarefa 4

- Criar uma *navigation bar* com elemento HTML *unordered list* (`<ul>`)
 - *Unordered list* composta por items (`<li>`) e hyperlinks (`<a>`);
 - Cor Azul: `#444bf6`
 - Cor Verde: `#5bcaa7`

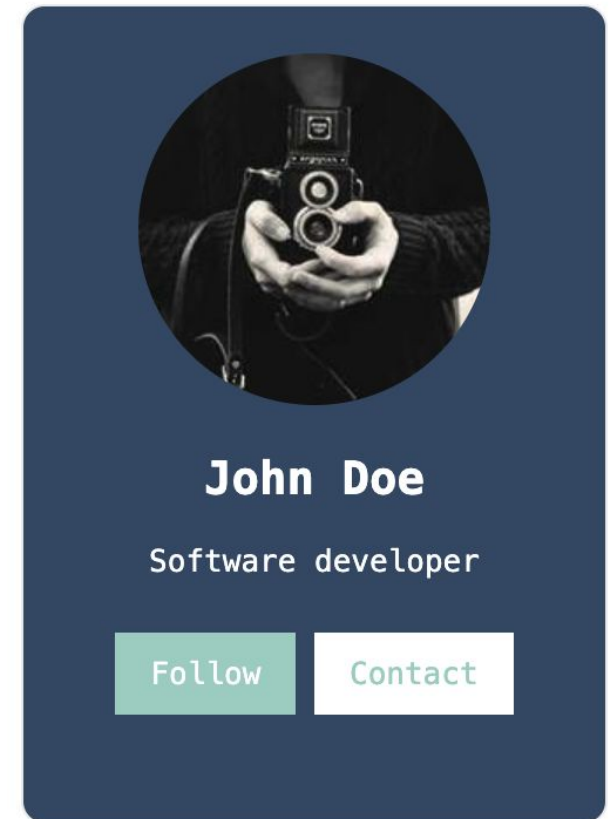
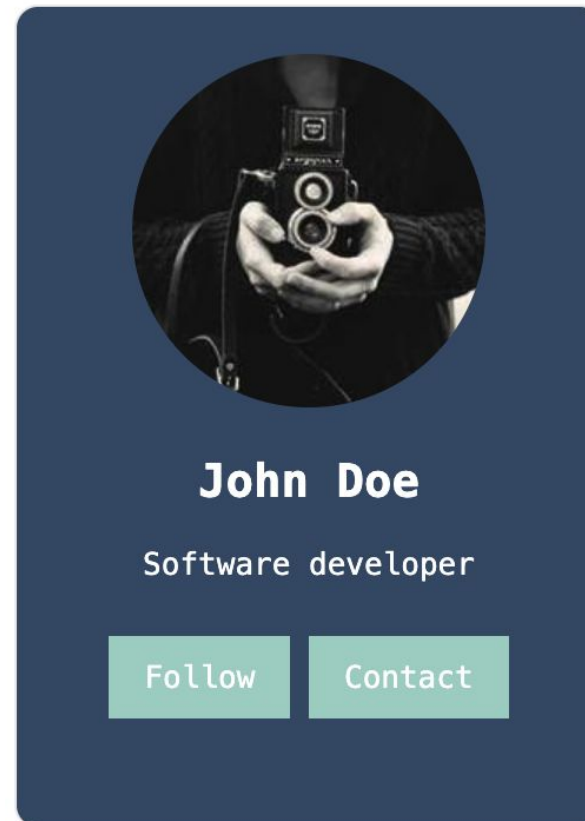


Exercício 2

- Criar um *User Profile Card*:
 - Background: #385170
 - Hyperlinks: #9fd3c7

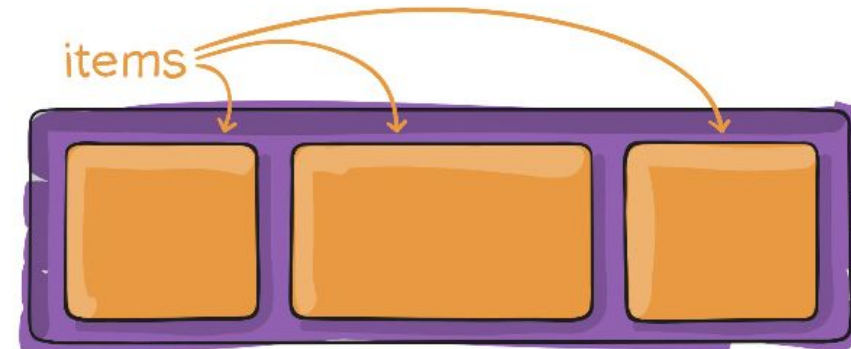
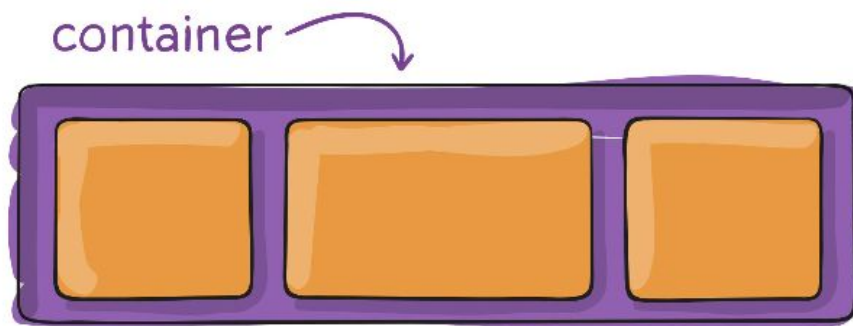
Dica: para a imagem usar um “container” e aplicar propriedade `background-image`

https://www.w3schools.com/cssref/pr_background-image.php



- O Flexbox é o modelo moderno para criação de layouts;
- Permite criar layouts de forma eficiente, alinhando e distribuindo os elementos dentro de uma caixa/container, mesmo quando a sua dimensão é desconhecida e dinâmica.
- O seu propósito principal é facilitar o posicionamento dos elementos de forma a preencher o espaço disponível da melhor forma possível.

- O Flexbox inclui todo um conjunto de propriedades, sendo que algumas são afetas à caixa exterior (flex container) e outras são direcionadas aos elementos internos (flex items)
- Para utilizar este modelo precisamos de definir o atributo **display: flex** no container;



Flexbox

Flex container pode ter as seguintes propriedades:

```
.container {  
  display: flex;  
}
```

- flex-direction
- flex-wrap
- flex-flow
- justify-content
- align-items
- align-content



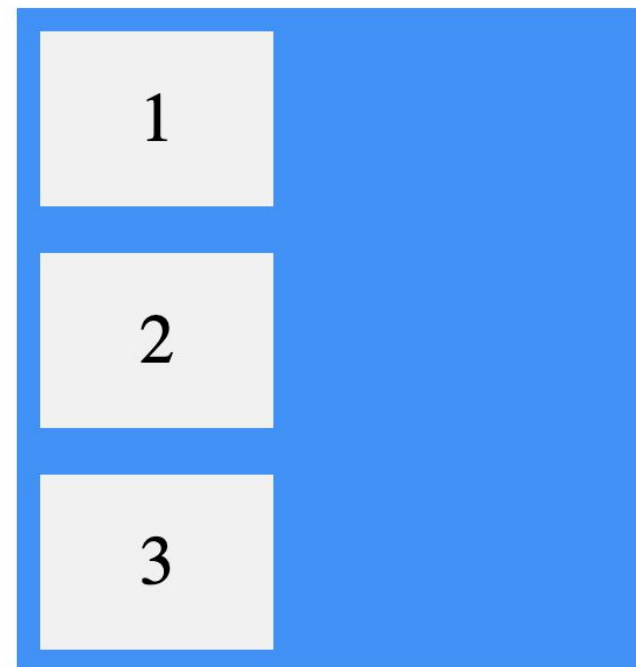
Flexbox

`flex-direction: row | column | row-reverse | column-reverse`

```
.container {  
  display: flex;  
  flex-direction: row;  
}
```



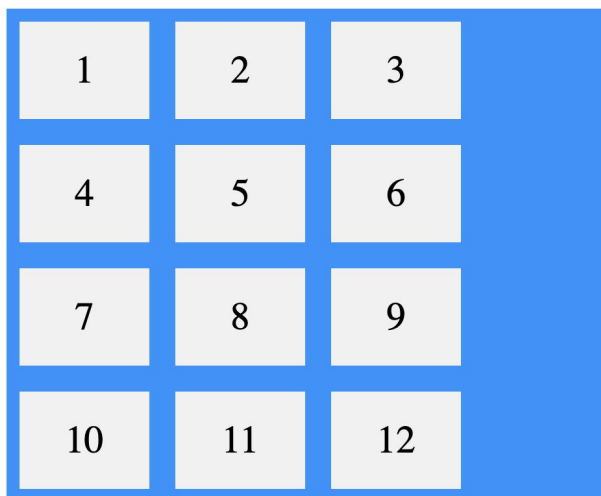
```
.container {  
  display: flex;  
  flex-direction: column;  
}
```



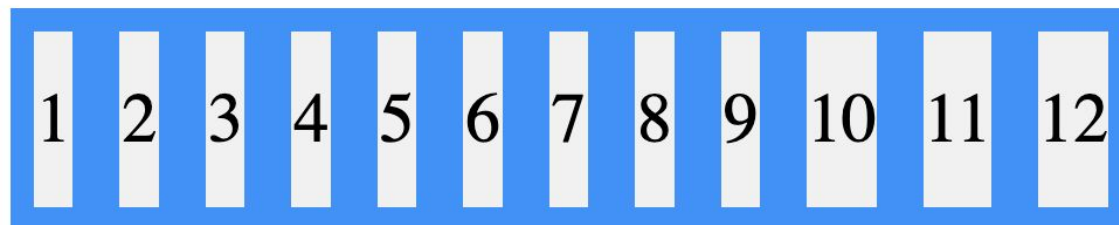
Flexbox

`flex-wrap: wrap | nowrap (default)`

```
.container {  
  display: flex;  
  flex-wrap: wrap;  
}
```



```
.container {  
  display: flex;  
  flex-wrap: nowrap;  
}
```



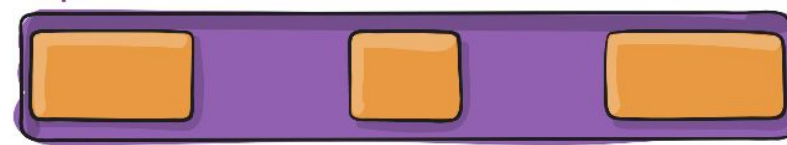
Flexbox

justify-content

flex-start



space-between



flex-end



space-around



center



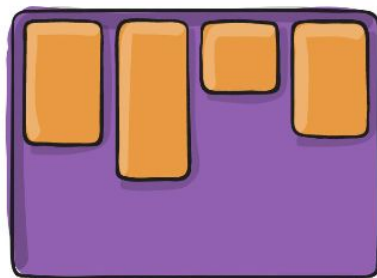
space-evenly



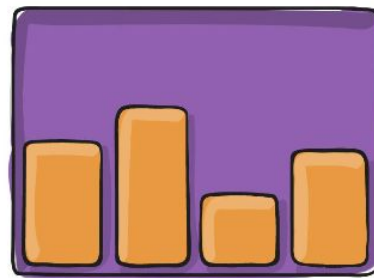
Flexbox

align-items

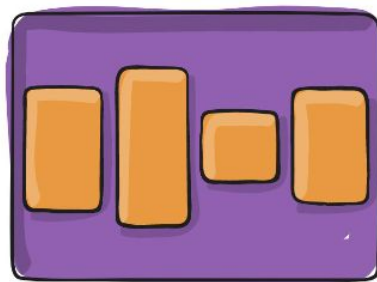
flex-start



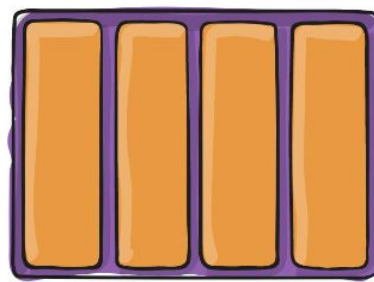
flex-end



center



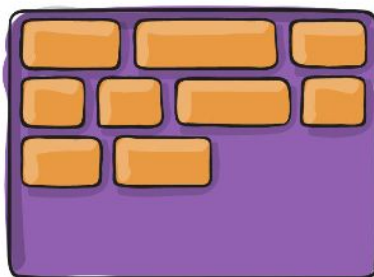
stretch



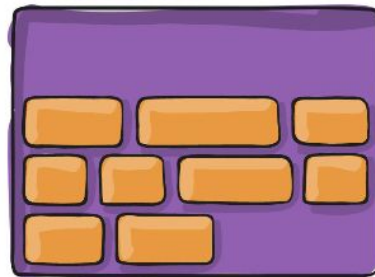
Flexbox

align-content (apenas quando é utilizado o flex-wrap)

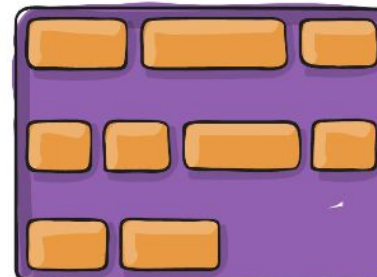
flex-start



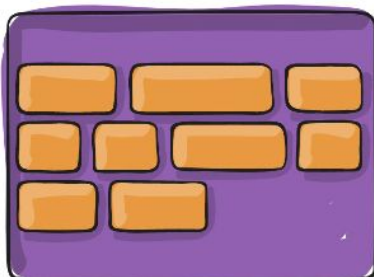
flex-end



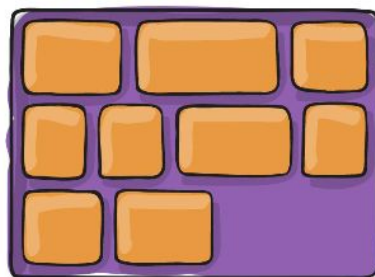
space-between



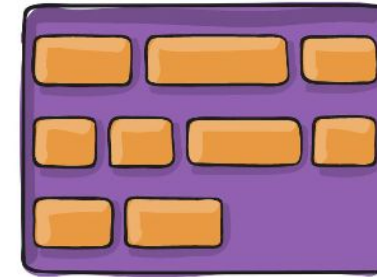
center



stretch



space-around



Flexbox - items

- Os elementos diretos de um container *flex*, são automaticamente classificados em *flex items*
- Flex items podem ter as seguintes propriedades;
 - order
 - flex-grow
 - flex-shrink
 - flex-basis
 - flex
 - align-self

```
<div class="flex-container">
  <div>1</div>
  <div>2</div>
  <div>3</div>
  <div>4</div>
</div>
```

Flexbox - items

- order

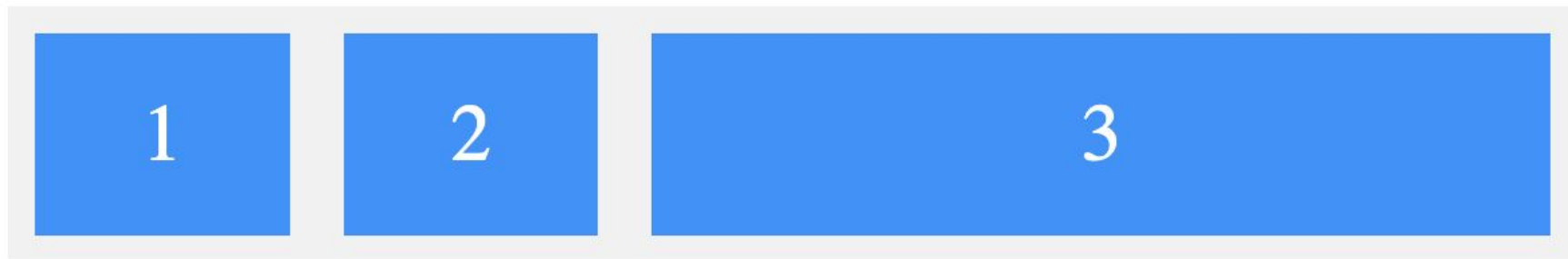
```
<div class="flex-container">  
  <div style="order: 3">1</div>  
  <div style="order: 2">2</div>  
  <div style="order: 4">3</div>  
  <div style="order: 1">4</div>  
</div>
```



Flexbox - items

- **flex-grow:** especifica quanto um elemento cresce relativo aos outros elementos; Elementos ocupam o tamanho máximo do container.

```
<div class="flex-container">
  <div style="flex-grow: 1">1</div>
  <div style="flex-grow: 1">2</div>
  <div style="flex-grow: 4">3</div>
</div>
```

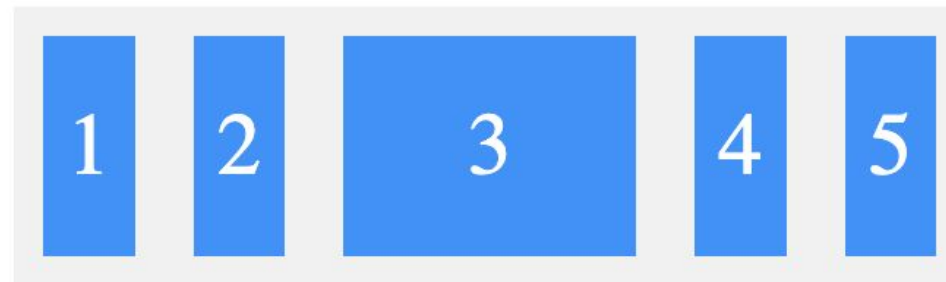


Flexbox - items

- **flex-shrink**: especifica quanto um elemento pode encolher caso não exista espaço para todos os elementos.

```
<div class="flex-container">
  <div>1</div>
  <div>2</div>
  <div style="flex-shrink: 0">3</div>
  <div>4</div>
  <div>5</div>
</div>
```

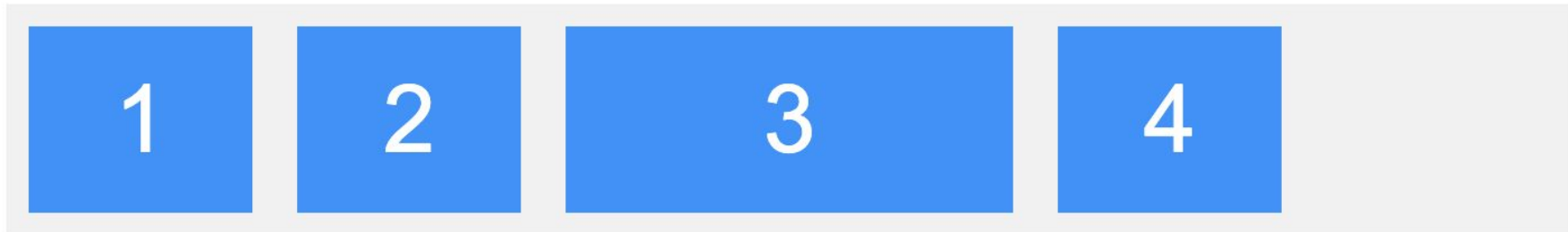
```
.flex-container div {
  width: 100px;
}
```



Flexbox - items

- **flex-basis**: especifica o tamanho inicial do elemento.

```
<div class="flex-container">
  <div>1</div>
  <div>2</div>
  <div style="flex-basis: 200px">3</div>
  <div>4</div>
</div>
```

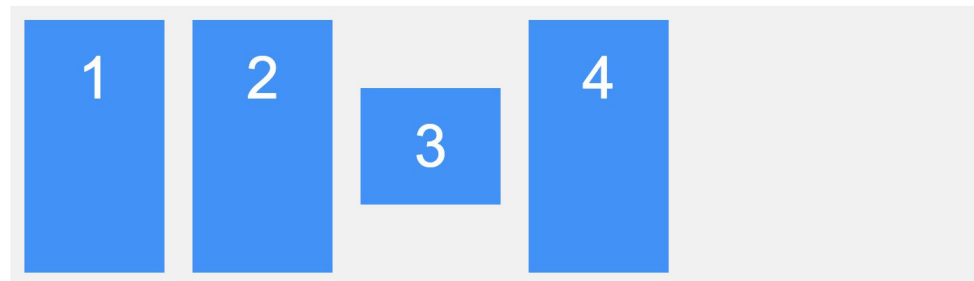


Flexbox - items

- **align-self**: especifica o alinhamento para o elemento

```
<div class="flex-container">
  <div>1</div>
  <div>2</div>
  <div style="align-self: center">3</div>
  <div>4</div>
</div>
```

```
.flex-container {
  display: flex;
  height: 200px;
}
```



Exercício

- box-shadow
- Google Font (para o título)
- background-image
- Flexbox containers
- transition: change background color of button on hover

#e97132ff

FOR SALE

#757575

Consultar



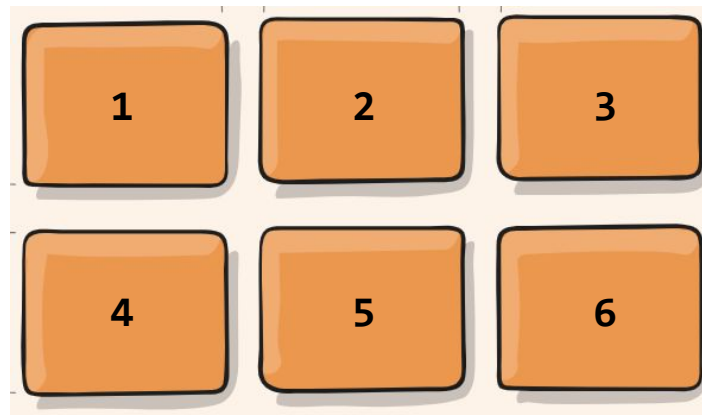
Display: grid

Usado para criar *layouts* modernos e flexíveis.

Sistema de layout baseado em **grelha**: com linhas e colunas facilitando o design de páginas web

```
<div class="grid-container">
  <div class="item1">1</div>
  <div class="item2">2</div>
  <div class="item3">3</div>
  <div class="item4">4</div>
  <div class="item5">5</div>
  <div class="item6">6</div>
</div>
```

```
.grid-container {
  display: grid;
}
```



Display: grid

Definimos a propriedade display com o valor grid no elemento *parent*

Os elementos diretos (*child*) tornam-se automaticamente *itens grid*

```
.grid-container {
  display: grid;
  grid-template-columns: auto auto auto;
  background-color: #2196F3;
  padding: 10px;
}
```

1	2	3
4	5	6

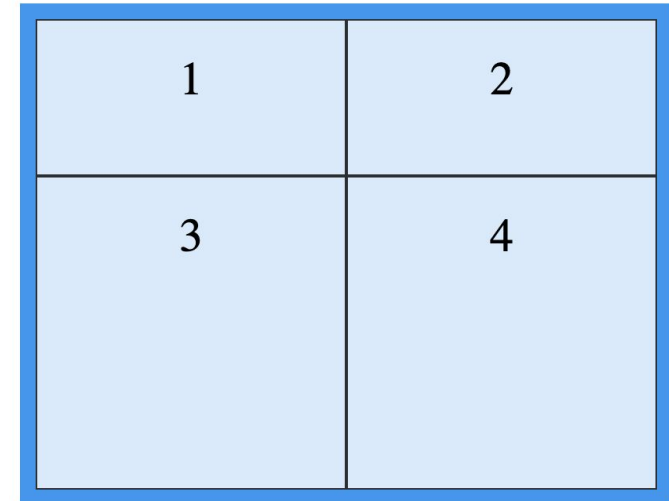
Display: grid

Definir linhas e colunas

grid-template-columns ; **grid-template-rows**

Ao definir o elemento *parent* como um grid, podemos especificar quantas colunas e linhas ele terá (e os tamanhos de cada linha/coluna)

```
.grid-container {
  display: grid;
  grid-template-columns: 1fr 1fr;
  grid-template-rows: 100px
  200px;
}
```



2 colunas que ocupam a mesma fração do espaço disponível (1 fr: uma fração do espaço);

2 linhas, uma com 100px de altura e a segunda com 200px;

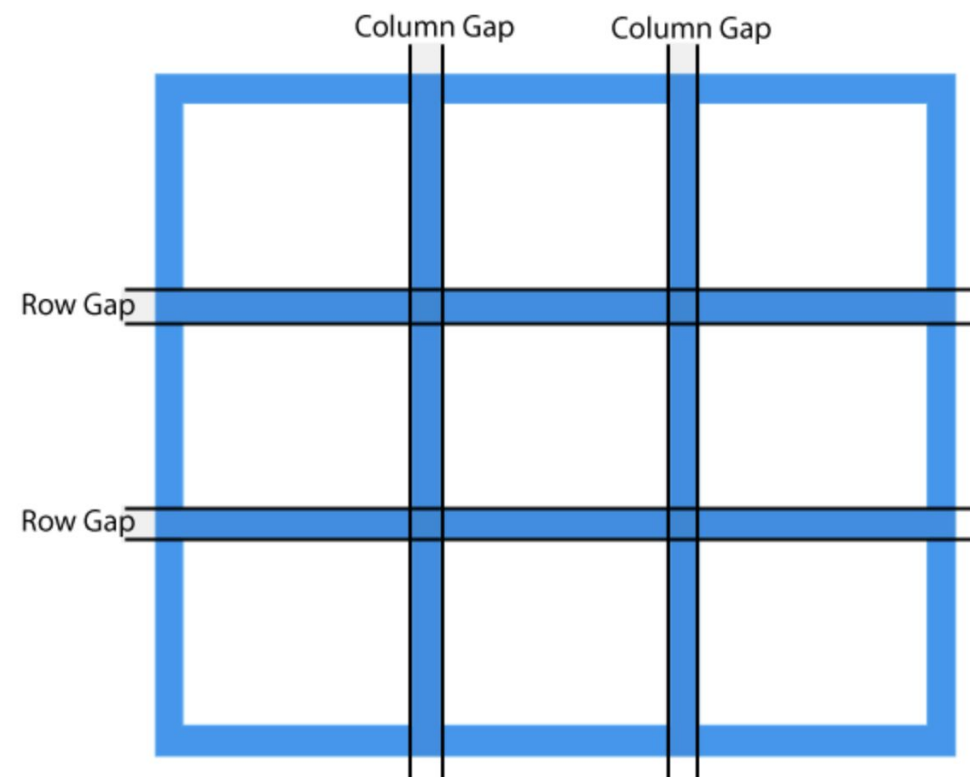
Display: grid

Espaçamento entre linhas e colunas

column-gap ; **row-gap** ; **gap**

- Os espaços entre cada coluna/linha são chamados *gaps*
- Podemos ajustar o tamanho do “gap” através destas 3 propriedades

```
.grid-container {  
  display: grid;  
  grid-template-columns: 1fr 1fr 1fr;  
  gap: 20px;  
}
```

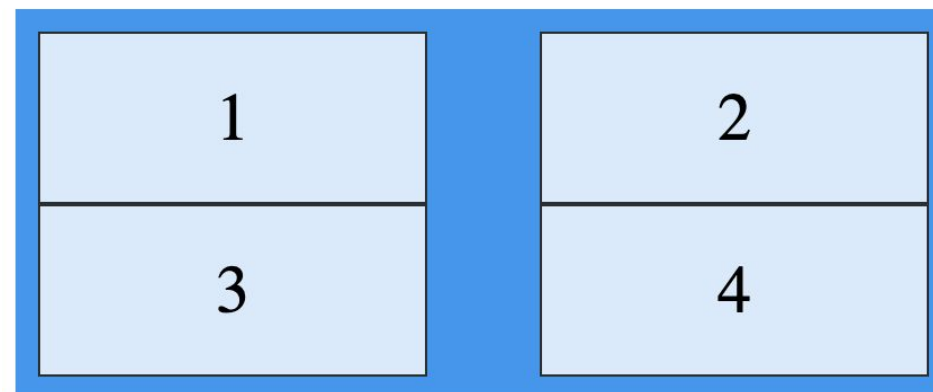


Display: grid

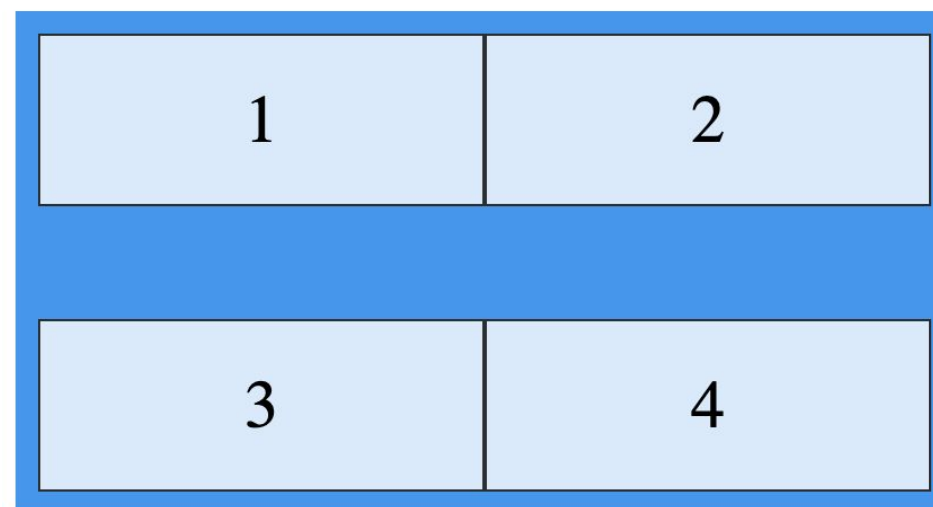
Espaçamento entre linhas e colunas

column-gap ; **row-gap** ; **gap**

```
.grid-container {  
  display: grid;  
  grid-template-columns: 1fr 1fr;  
  column-gap: 20px;  
}
```



```
.grid-container {  
  display: grid;  
  grid-template-columns: 1fr 1fr;  
  row-gap: 20px;  
}
```



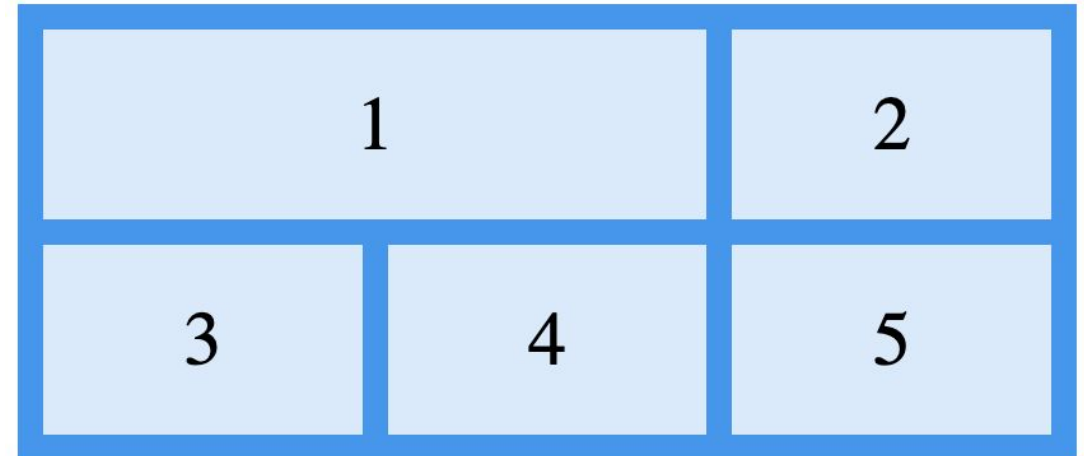
Display: grid

Posicionamento dos elementos dentro do *container*

grid-column ; **grid-row**

```
.grid-container {  
  display: grid;  
  grid-template-columns: auto auto auto;  
  gap: 10px;  
}
```

```
.item1 {  
  grid-column-start: 1;  
  grid-column-end: 3;  
}
```



grid-column: 1 / 3;

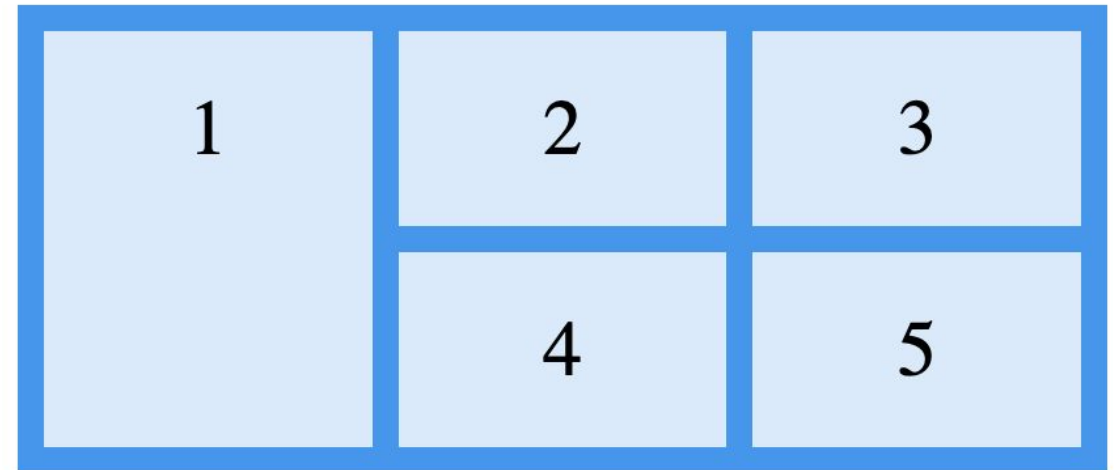
Display: grid

Posicionamento dos elementos dentro do *container*

grid-column ; **grid-row**

```
.grid-container {
  display: grid;
  grid-template-columns: auto auto auto;
  gap: 10px;
}
```

```
.item1 {
  grid-row-start: 1;
  grid-row-end: 3;
}
```



grid-row: 1 / 3;

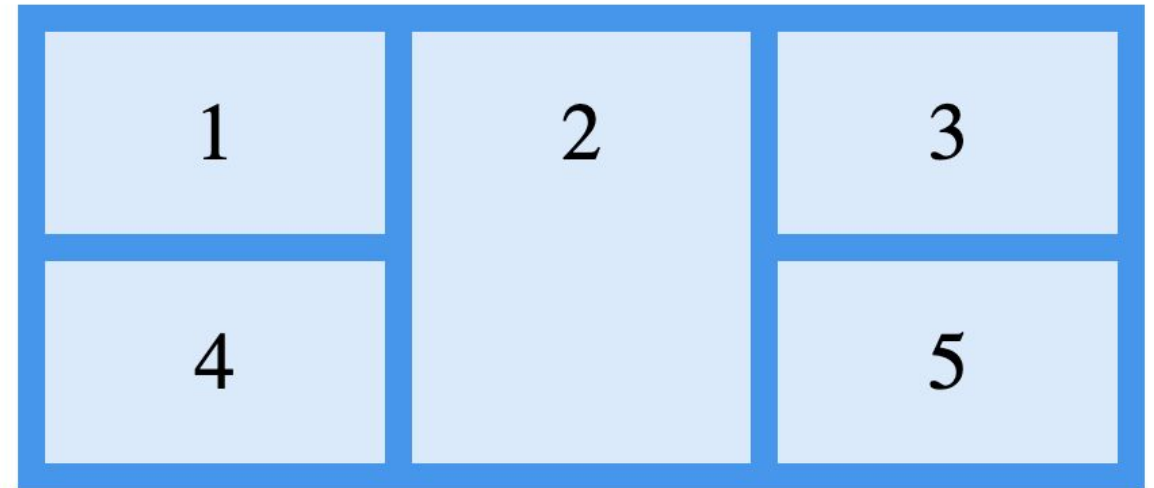
Display: grid

Posicionamento dos elementos dentro do *container*

grid-column ; **grid-row**

```
.grid-container {  
  display: grid;  
  grid-template-columns: auto auto auto;  
  gap: 10px;  
}
```

```
.item2 {  
  grid-row-start: 1;  
  grid-row-end: 3;  
  grid-column: 2 / 3;  
}
```



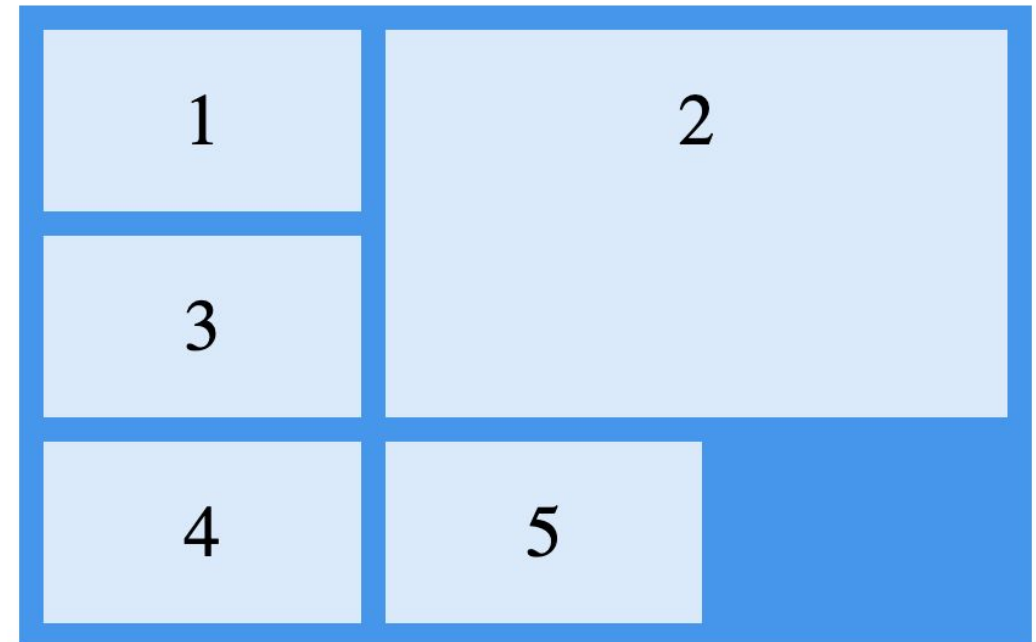
Display: grid

Posicionamento dos elementos dentro do *container*

grid-column ; **grid-row**

```
.grid-container {
  display: grid;
  grid-template-columns: auto auto auto;
  gap: 10px;
}
```

```
.item2 {
  grid-row-start: 1;
  grid-row-end: 3;
  grid-column: 2 / 4;
}
```

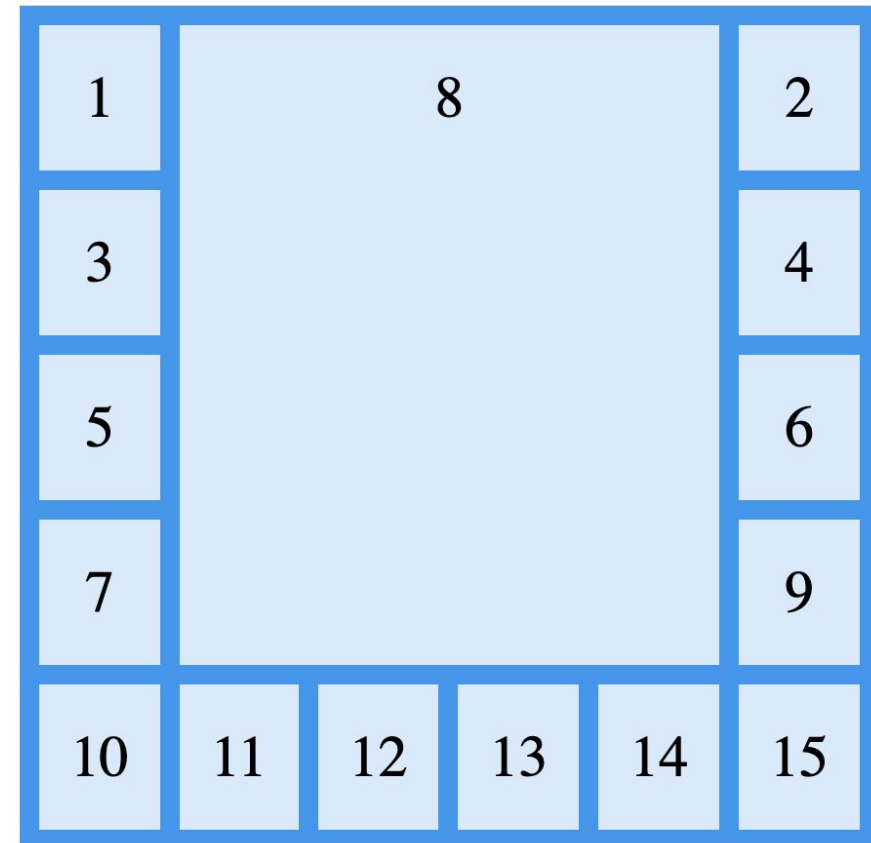


Display: grid

grid-area grid-row-start / grid-column-start / grid-row-end / grid-column-end

```
.grid-container {
  display: grid;
  grid-template-columns: repeat(6, auto);
  gap: 10px;
}

.item8 {
  grid-area: 1 / 2 / 5 / 6;
}
```



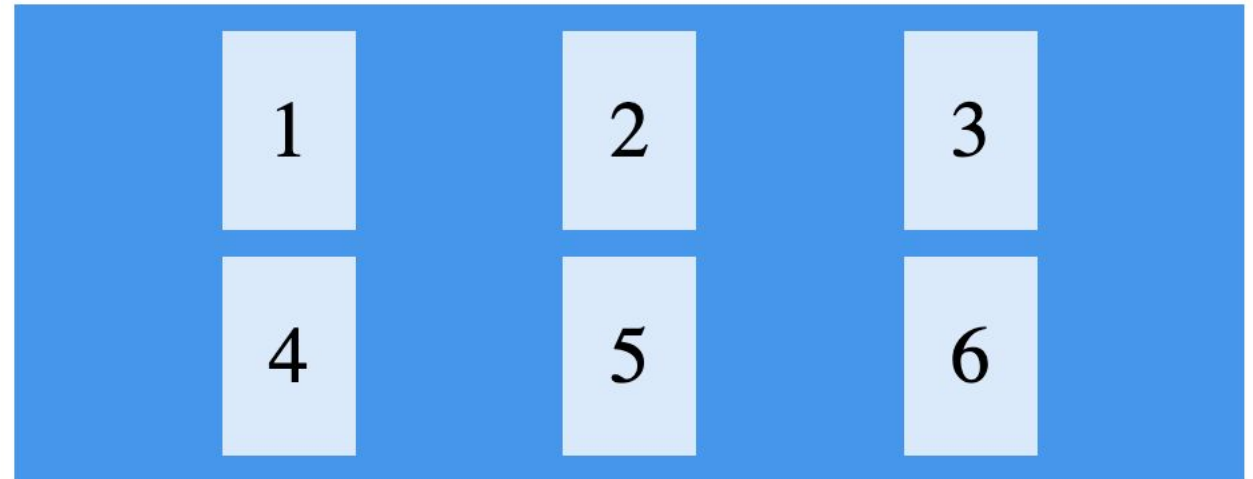
Display: grid

justify-content space-evenly / space-around / space-between / center / start / end

Alinhar **horizontalmente** os elementos dentro do container *grid*

```
.grid-container {
  display: grid;
  grid-template-columns: 50px 50px 50px;
  gap: 10px;
  background-color: #2196F3;
  padding: 10px;
  justify-content: space-evenly;
}

.grid-container > div {
  background-color: rgba(255, 255, 255, 0.8);
  text-align: center;
  padding: 20px 0;
  text-align: 30px;
}
```



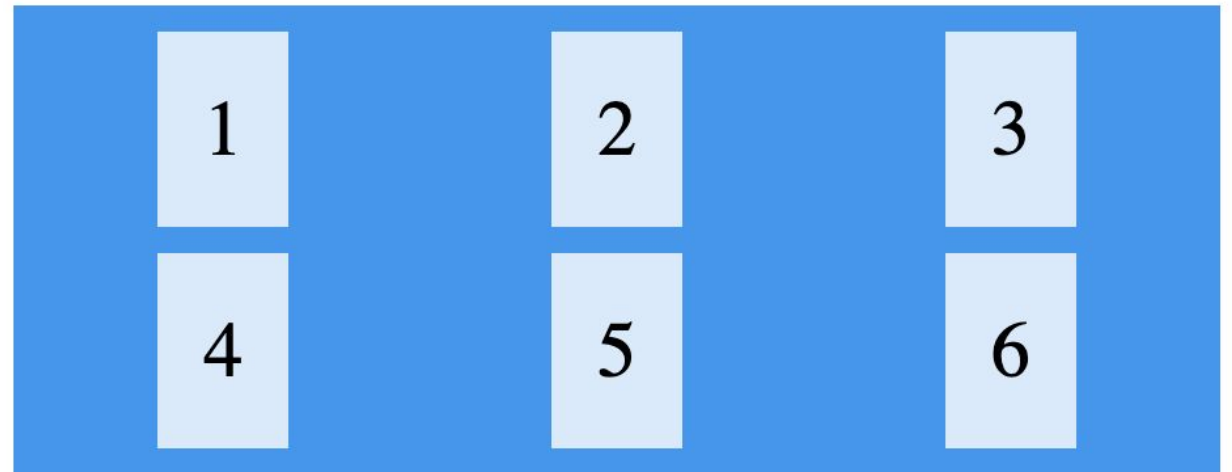
Display: grid

justify-content space-evenly / space-around / space-between / center / start / end

Alinhar **horizontalmente** os elementos dentro do container *grid*

```
.grid-container {
  display: grid;
  grid-template-columns: 50px 50px 50px;
  gap: 10px;
  background-color: #2196F3;
  padding: 10px;
  justify-content: space-around;
}

.grid-container > div {
  background-color: rgba(255, 255, 255, 0.8);
  text-align: center;
  padding: 20px 0;
  text-align: 30px;
}
```



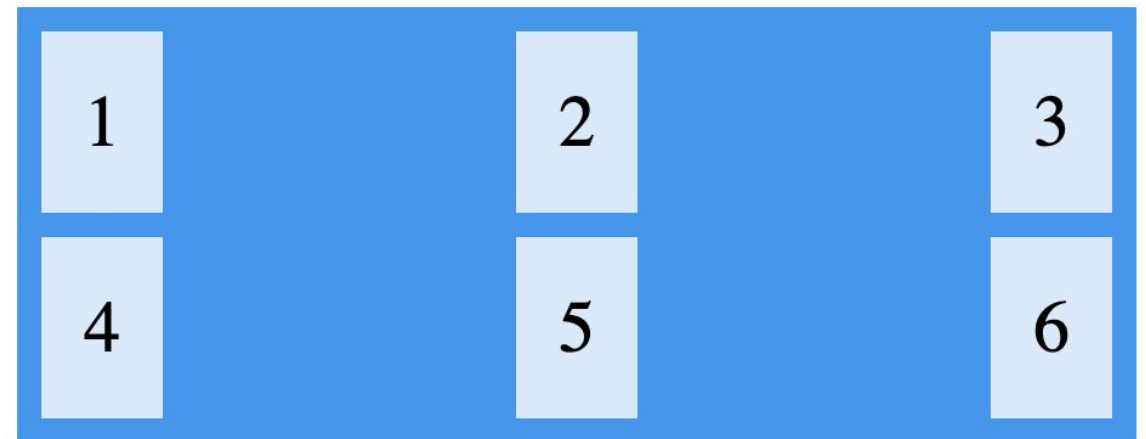
Display: grid

justify-content space-evenly / space-around / **space-between** / center / start / end

Alinhar **horizontalmente** os elementos dentro do container *grid*

```
.grid-container {
  display: grid;
  grid-template-columns: 50px 50px 50px;
  gap: 10px;
  background-color: #2196F3;
  padding: 10px;
  justify-content: space-between;
}

.grid-container > div {
  background-color: rgba(255, 255, 255, 0.8);
  text-align: center;
  padding: 20px 0;
  text-align: 30px;
}
```



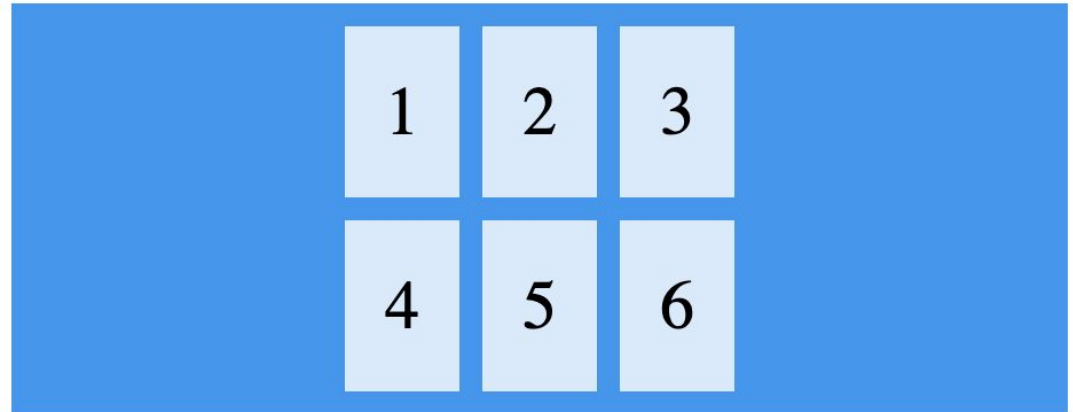
Display: grid

justify-content space-evenly / space-around / space-between / center / start / end

Alinhar **horizontalmente** os elementos dentro do container *grid*

```
.grid-container {
  display: grid;
  grid-template-columns: 50px 50px 50px;
  gap: 10px;
  background-color: #2196F3;
  padding: 10px;
  justify-content: center;
}

.grid-container > div {
  background-color: rgba(255, 255, 255, 0.8);
  text-align: center;
  padding: 20px 0;
  text-align: 30px;
}
```



Display: grid

justify-content space-evenly / space-around / space-between / center / **start** / end

Alinhar **horizontalmente** os elementos dentro do container *grid*

```
.grid-container {
  display: grid;
  grid-template-columns: 50px 50px 50px;
  gap: 10px;
  background-color: #2196F3;
  padding: 10px;
  justify-content: start;
}

.grid-container > div {
  background-color: rgba(255, 255, 255, 0.8);
  text-align: center;
  padding: 20px 0;
  text-align: 30px;
}
```



Display: grid

justify-content space-evenly / space-around / space-between / center / start / end

Alinhar **horizontalmente** os elementos dentro do container *grid*

```
.grid-container {
  display: grid;
  grid-template-columns: 50px 50px 50px;
  gap: 10px;
  background-color: #2196F3;
  padding: 10px;
  justify-content: end;
}

.grid-container > div {
  background-color: rgba(255, 255, 255, 0.8);
  text-align: center;
  padding: 20px 0;
  text-align: 30px;
}
```

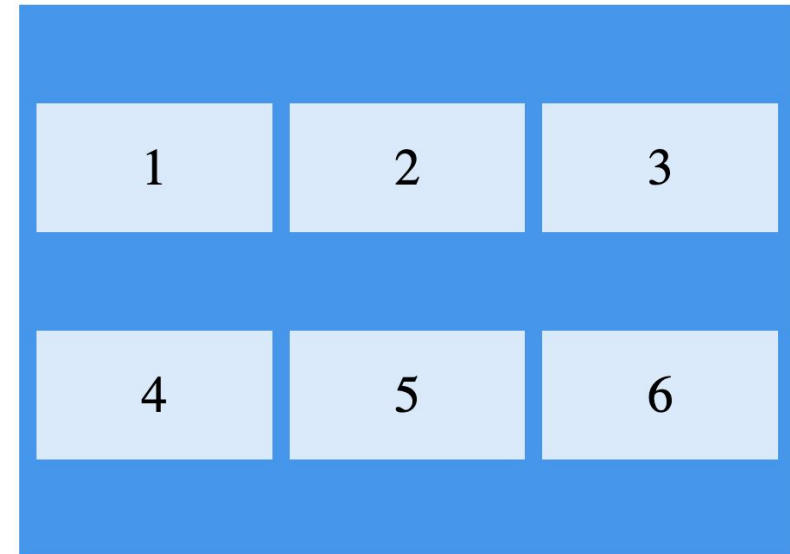


Display: grid

align-content space-evenly / space-around / space-between / center / start / end

Alinhar **verticalmente** os elementos dentro do container *grid*

```
.grid-container {
  display: grid;
  height: 400px;
  grid-template-columns: auto auto auto;
  gap: 10px;
  background-color: #2196F3;
  padding: 10px;
  align-content: space-evenly;
}
```

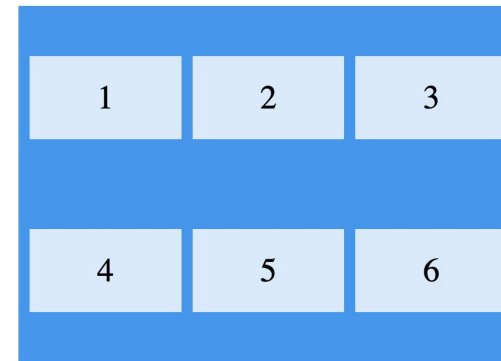


Display: grid

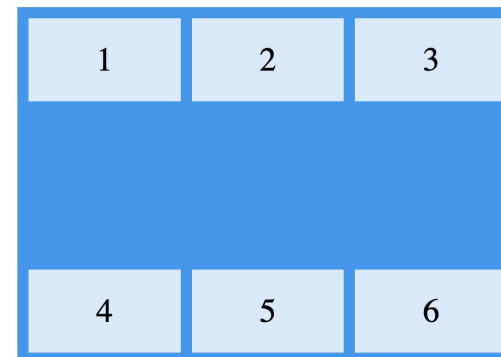
align-content space-evenly / space-around / space-between / center / start / end

Alinhar **verticalmente** os elementos dentro do container *grid*

```
align-content: space-around;
```



```
align-content: space-between;
```



Display: grid

align-content space-evenly / space-around / space-between / center / start / end

Alinhar **verticalmente** os elementos dentro do container *grid*

`align-content: center;`

1	2	3
4	5	6

`align-content: start;`

1	2	3
4	5	6

`align-content: end;`

1	2	3
4	5	6

Exercício

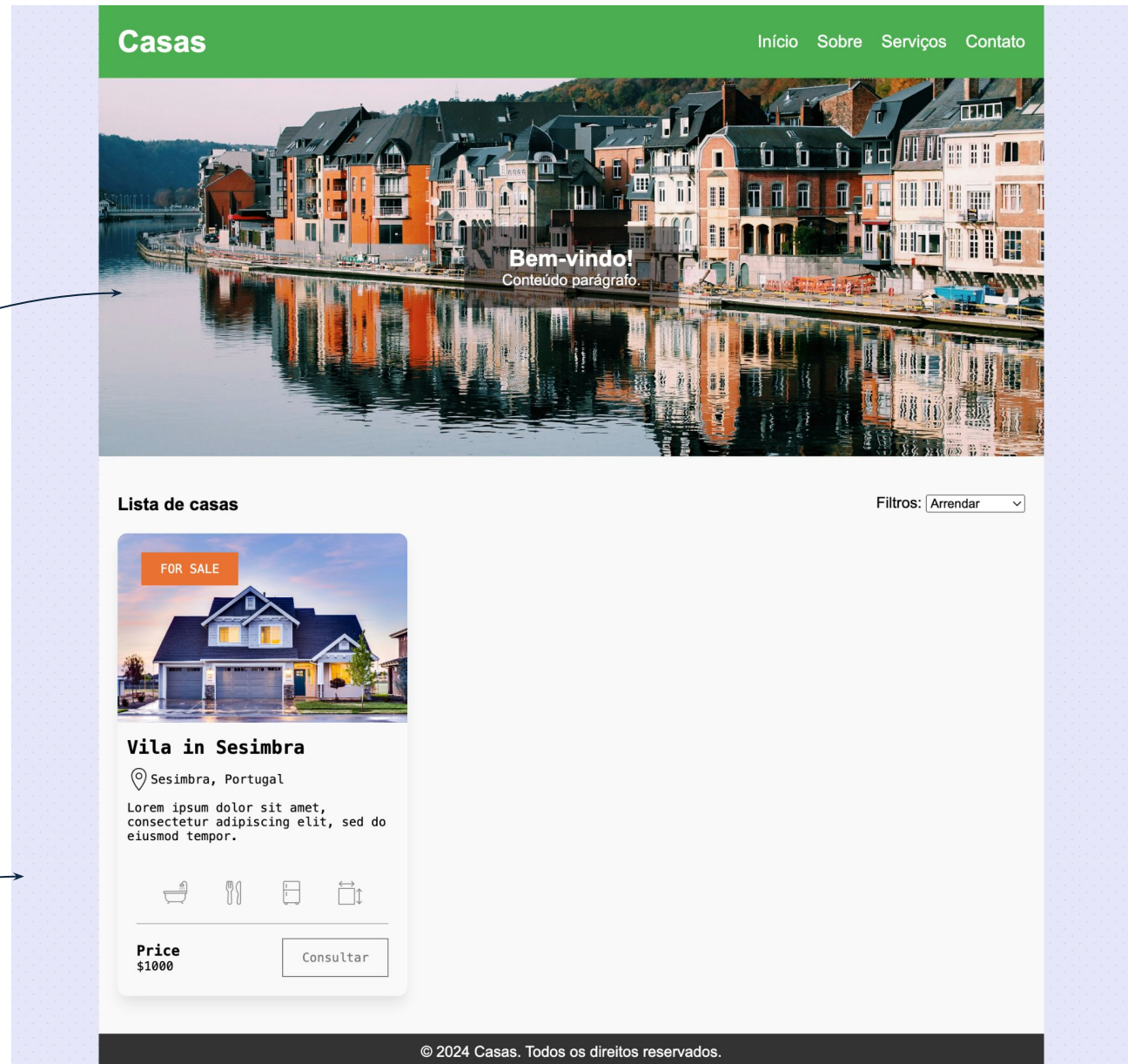
Home Page

- Container com header, main e footer, usando propriedade display: grid;
- Header color: #4CAF50
- Footer color: #333333

Section com background-image e elementos posicionados ao centro com Flexbox

body com background-image

max-width: 1000px

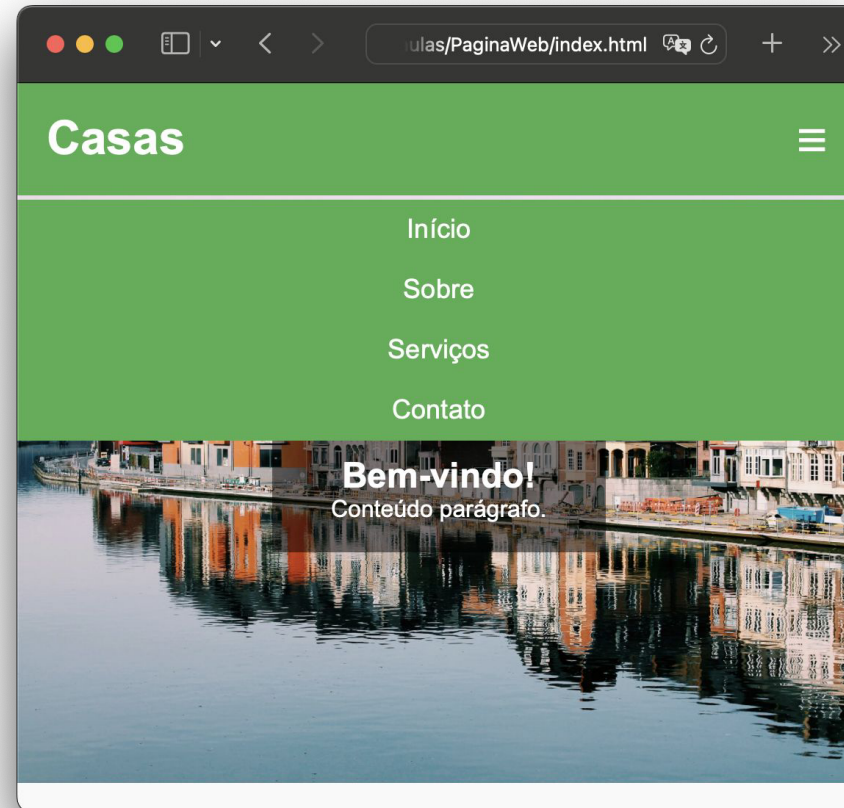


Exercício

`<nav>`, com class: “nav”
`<ul>`, com class: “nav-list”

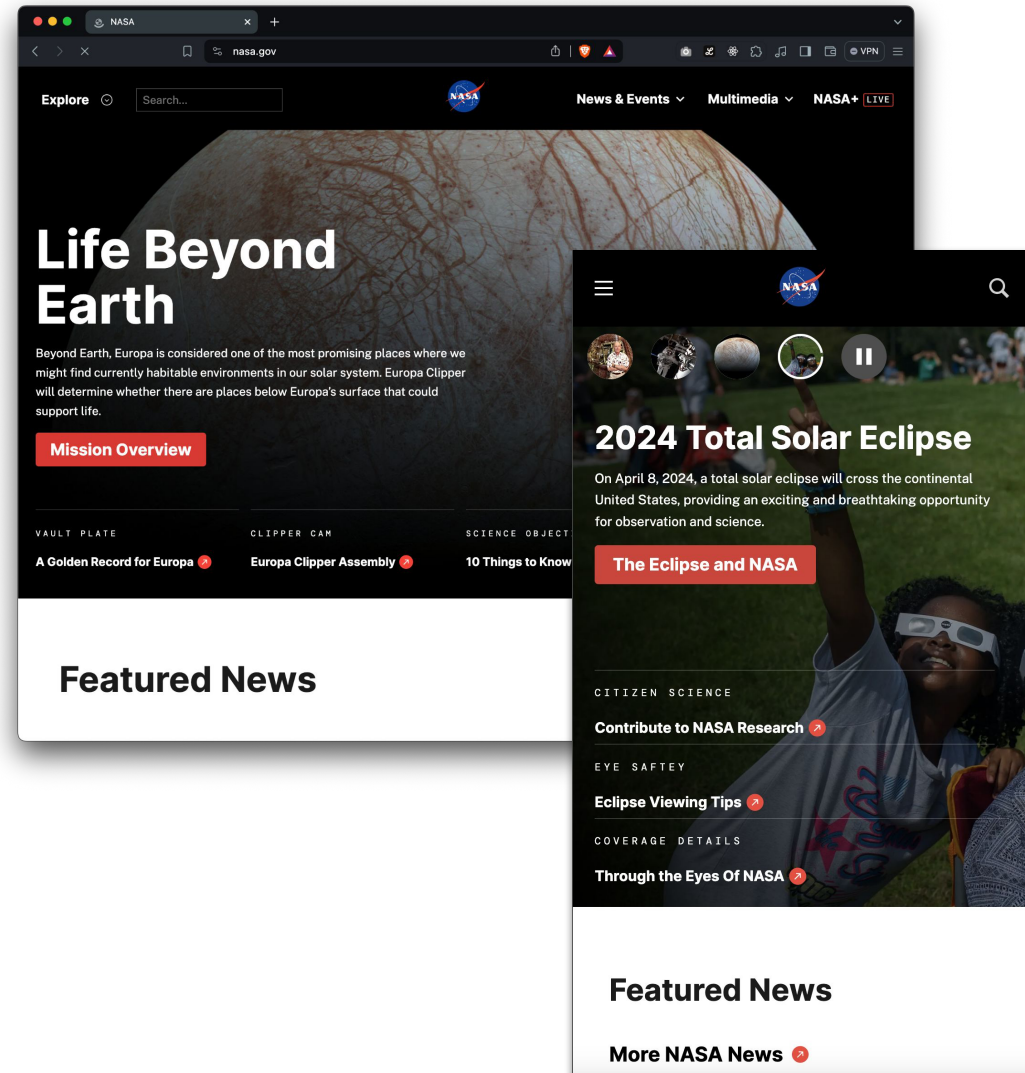
Incluir o seguinte `<div>` dentro do `<header>`

```
<div class="menu-toggle" id="menu-toggle">  
  ≡  
</div>
```



Media Queries

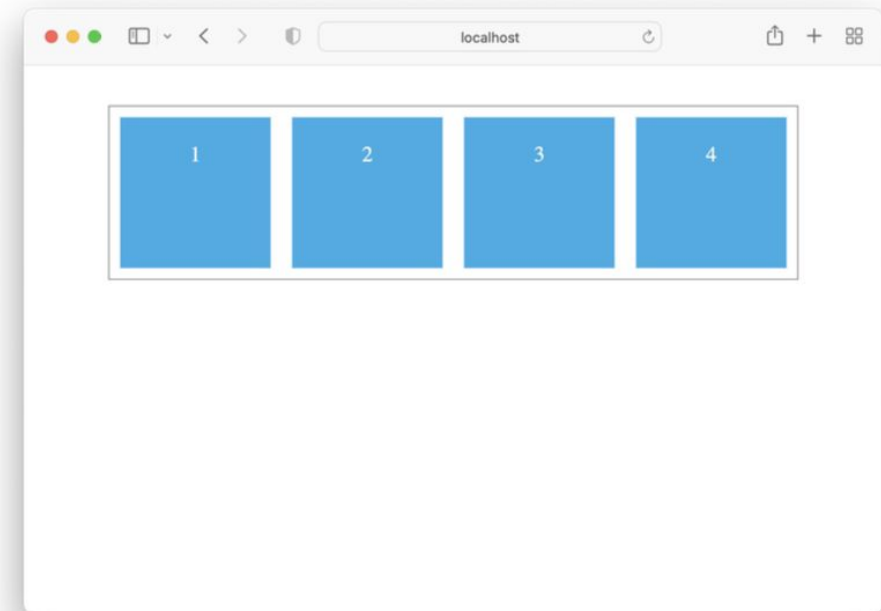
- Como podemos programar a nossa página para aparecer bem em todos os ecrãs?
- Utilizar larguras máximas não é suficiente e é necessário definir condições concretas para como o layout é apresentado em ecrãs de dimensões diferentes.



Media Queries

- É habitual que uma página apresente uma disposição horizontal de elementos em ecrãs mais largos.
- Como podemos alinhar aqueles elementos verticalmente quando o ecrã fica menos largo?

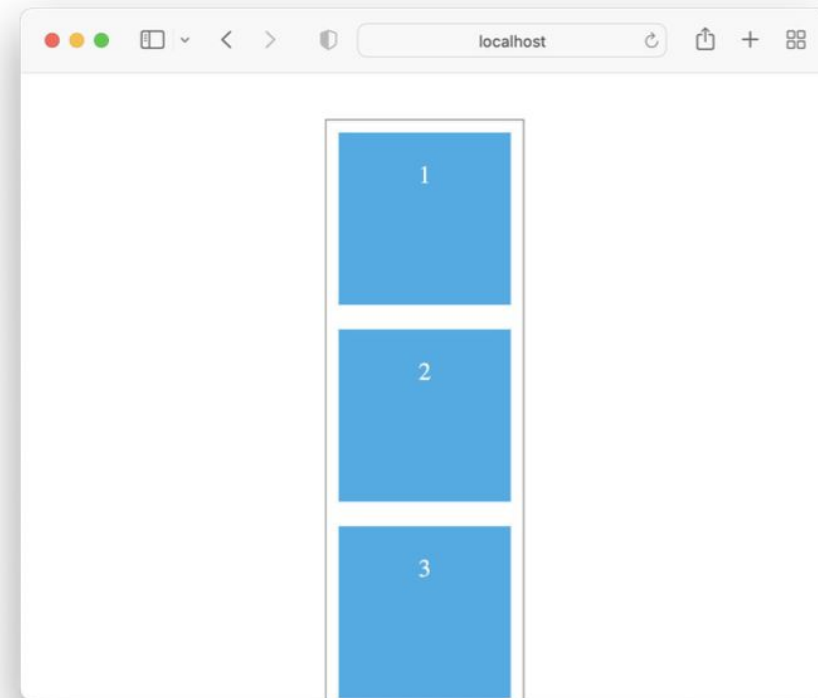
```
.container {  
  position: relative;  
  display: flex;  
  border: #8B8B8B solid 1px;  
  margin: 30px;  
  gap: 10px;  
  padding: 10px;  
}
```



Media Queries

```
.container {  
  position: relative;  
  display: flex;  
  border: #8B8B8B solid 1px;  
  margin: 30px;  
  gap: 10px;  
  padding: 10px;  
}  
  
@media (max-width: 768px) {  
  .container {  
    flex-direction: column;  
  }  
}
```

width <= 768px



Media Queries

- Existe um conjunto universal de valores que definem as quebras nas páginas

Dispositivos muito pequenos (600px ou menos)	<code>@media screen and (max-width: 600px) { . . . }</code>
Dispositivos pequenos (600px ou mais)	<code>@media screen and (min-width: 600px) { . . . }</code>
Dispositivos médios (768px ou mais)	<code>@media screen and (min-width: 768px) { . . . }</code>
Dispositivos grandes (992px ou mais)	<code>@media screen and (min-width: 992px) { . . . }</code>
Dispositivos muito grandes (1200px ou mais)	<code>@media screen and (min-width: 1200px) { . . . }</code>

Media Queries

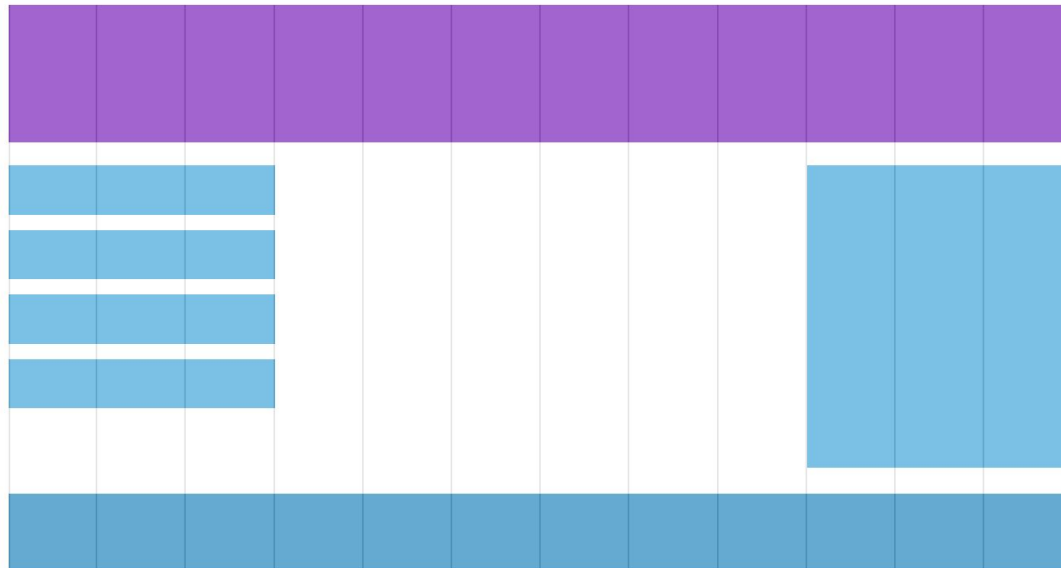
- Para a página ser corretamente apresentada nos dispositivos móveis é necessário adicionar a seguinte tag <meta> dentro do head:

```
<meta name="viewport" content="width=device-width, initial-scale=1.0">
```

- Caso esta linha não seja utilizada, o dispositivo vai considerar uma largura de janela maior e poderá não aplicar as regras de responsabilidade.

Media Queries

- Para simplificar o sistema, podemos dividir a página em colunas - **Grid-view**.



```
.col-1 {width: 8.33%;}  
.col-2 {width: 16.66%;}  
.col-3 {width: 25%;}  
.col-4 {width: 33.33%;}  
.col-5 {width: 41.66%;}  
.col-6 {width: 50%;}  
.col-7 {width: 58.33%;}  
.col-8 {width: 66.66%;}  
.col-9 {width: 75%;}  
.col-10 {width: 83.33%;}  
.col-11 {width: 91.66%;}  
.col-12 {width: 100%;}
```

- Cross-browser: garantir a compatibilidade com todos os dispositivos e com diferentes browsers. Muitas vezes temos de implementar código diferente para diferentes browsers.
- Responsive Web Design: O layout do website deve adaptar-se automaticamente aos diferentes tamanhos de ecrã dos dispositivos; Muitas vezes temos de mudar o layout da página ou remover texto/images;
- Performance: Páginas Web devem ser rápidas carregar. Escolher tamanho ideal de imagens; minimizar o código; etc.
- Accessibility: Website deve ser acessível a todos - adaptar conteúdo a pessoas com deficiências visuais, deficiências auditivas, incapacidades cognitivas ou incapacidades físicas.
- Internationalization
- Privacy & Security

- Usar elementos HTML de forma a fornecer significado estrutural e contextual para o conteúdo da página.
- Em vez de simplesmente usar elementos HTML com base na sua aparência visual, selecionar cuidadosamente os elementos que melhor descrevem a estrutura e conteúdo. - **HTML Semantic**
- Elementos que descrevem melhor o conteúdo: `<header>`, `<nav>`, `<article>`, `<section>`, `<footer>` - em vez de usar só `<div>` para todo o conteúdo
- SEO: mecanismos de busca utilizam a semântica HTML apropriados para entender melhor o contexto do conteúdo e melhor no posicionamento dos resultados de pesquisa
- Cada elemento tem um propósito semântico claro que ajuda na manutenção do website

Práticas Web - exemplo

<header>

<section>

<footer>

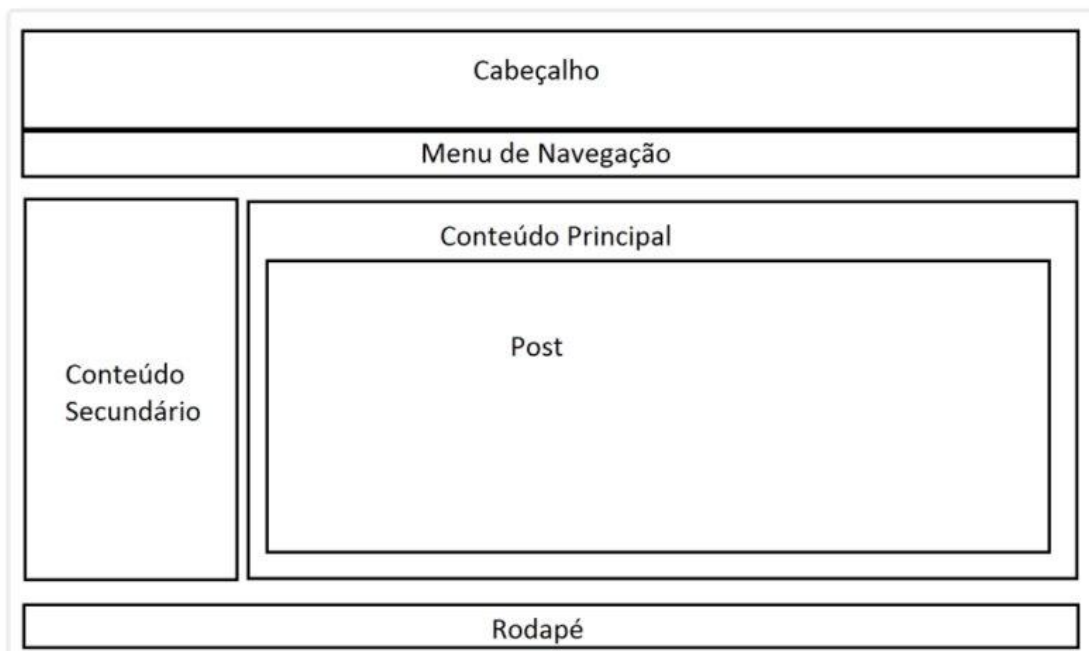


```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>Exemplo de HTML Semântico</title>
</head>
<body>
  <header>
    <h1>Meu Site</h1>
    <nav>
      <ul>
        <li><a href="#">Início</a></li>
        <li><a href="#">Sobre</a></li>
        <li><a href="#">Contato</a></li>
      </ul>
    </nav>
  </header>

  <section>
    <article>
      <h2>Título do Artigo</h2>
      <p>Conteúdo do artigo...</p>
    </article>
  </section>

  <footer>
    <p>&copy; 2024 Meu Site. Todos os direitos reservados.</p>
  </footer>
</body>
</html>
```

Práticas Web

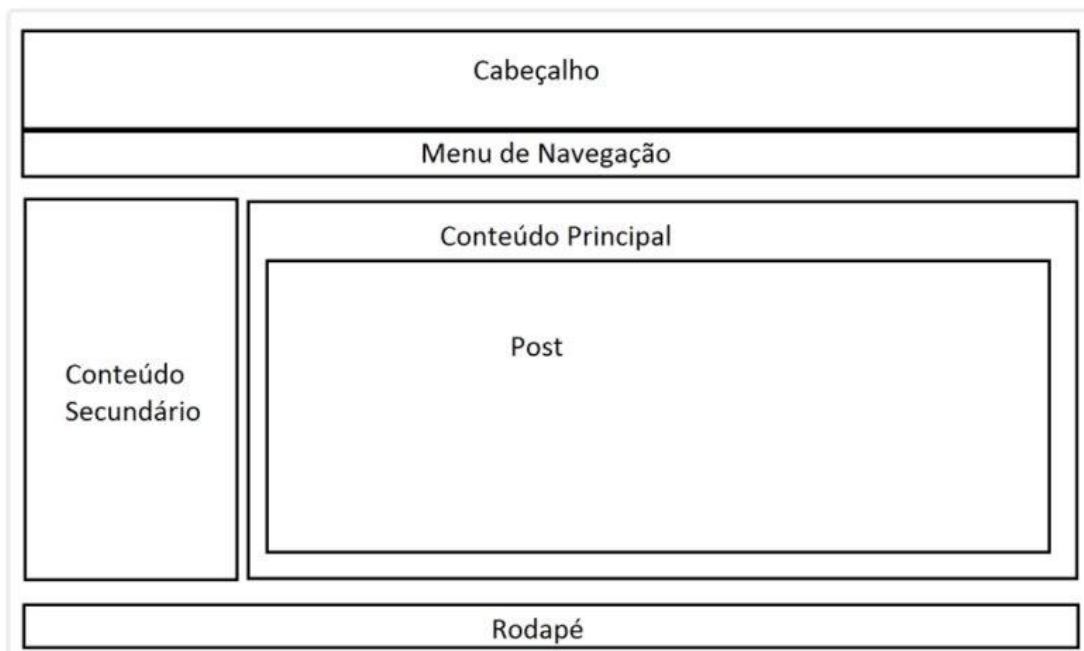


```
<div class="cabecalho" >
|   <div class="menu-navegacao" ></div>
</div>

<div class="conteudo-secundario" ></div>

<div class="conteudo-principal" >
|   <div class="post" ></div>
</div>

<div class="rodape" ></div>
```



```
<header class="cabecalho" >
|   <nav class="menu-navegacao" ></nav>
</header>

<aside class="conteudo-secundario" ></aside>

<main class="conteudo-principal" >
|   <article class="post" ></article>
</main>

<footer class="rodape" ></footer>
```

Práticas Web - Semântica

Div e Span: <div> não exibe nada, a menos que a usando CSS. As demais tags também são assim. Servem apenas para fazermos divisões.

- <div> é um elemento de bloco
- é um elemento de linha

Article

Define um artigo. Muito utilizado em Posts de blogs e sites de notícias.

Aside

Define um conteúdo ao lado do conteúdo da página. Muito utilizado em colunas ao lado da parte principal dos sites, onde normalmente possui a navegação, propagandas ou algo relacionado com o conteúdo principal.

Code

Define um pedaço de código de computador

Práticas Web - Semântica

Details e Summary

A tag <details> serve para acrescentar detalhes adicionais sobre algo, mas essa informação fica escondida, sendo exibida apenas quando o usuário clicar para ler.

A tag <summary> serve para exibir um título para o <details>, que ficará sempre visível.

Dialog

serve para criar modais (popups).

Footer

Define um rodapé para o documento ou seção.

Header

Define um cabeçalho para o documento ou seção.

Main

Define a parte principal de um documento. Por ser algo indicado como principal, o conteúdo dessa tag deve ser único e ela não deve ser filha de tags como <article>, <footer>, <header> ou <nav>.

Nav

Define um grupo de links de navegação. Essa tag é muito utilizada nos menus utilizados para se navegar entre as páginas de um site.

Section

Funciona como a Div, sendo mais específica. Define seções no documento, como capítulos, cabeçalhos, rodapés, etc.